TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP. HỒ CHÍ MINH (UTH)

KHOA CÔNG NGHỆ THÔNG TIN — BỘ MÔN CÔNG NGHỆ PHẦN MỀM

CẨM NANG HÀNH ĐỘNG & KỊCH BẢN VẤN ĐÁP ĐẠT ĐIỂM A

Học phần: CÔNG NGHỆ PHẦN MỀM (Mã HP: 122005 - 3 Tín chỉ) — Tích hợp Diễn giải Chi tiết Khái niệm Kỹ thuật

Giảng viên phụ trách: TS. Lê Văn Quốc Anh — ThS. Trần Thị Mỹ Tiên

Sinh viên thực hiện: Nhựt Phùng Minh (Backend 2 Lead — Dự án OISMS)

> **MỤC ĐÍCH TÀI LIỆU BỔ SUNG:** Tài liệu này không chỉ chuẩn hóa các tiêu chí Rubrics và kịch bản vấn đáp, mà còn **giải mã chi tiết, bình dân hóa toàn bộ các thuật ngữ kỹ thuật súc tích** (như WAC, Atomic Goods Receipt, Append-Only Ledger, Anti-Oversell, Advisory Lock, Cycle Detection, EAN-13, JSONB Signature, Token Rotation) bằng **ví dụ đời thường cụ thể**. Giúp sinh viên Nhựt Phùng Minh hiểu sâu bản chất, không bị bỡ ngỡ khi giảng viên hỏi vặn và tự tin đạt điểm tối đa (Mức A / 8.5 – 10).

I. TỔNG QUAN HỌC PHẦN & ĐỀ TÀI DỰ ÁN OISMS

1. Cơ cấu trọng số đánh giá học phần (3 Tín chỉ)

- **Đánh giá quá trình (50%):**
  - *Chuyên cần (Rubric A1.1, A1.2):* **10%**. **Quy định sống còn:** Vắng > 3 buổi = **CẤM THI**; -1 điểm cho mỗi buổi vắng.
  - *Bài tập thực hành trên máy (Rubric A3.1):* **10%**.
  - *Bài tập trên lớp / Thảo luận (Rubric A2.1):* **10%**.
  - *Thực hiện Bài tập lớn theo nhóm (Rubric A5.1, A5.2, A5.3):* **20%**.
- **Đánh giá kết thúc học phần (50%):**
  - *Báo cáo & Vấn đáp Bài tập lớn (Rubric A5.1, A5.4, A5.5):* **20%** (Chấm tập trung tuần 8 – 9). **Lưu ý:** Rớt Project = Cấm thi kết thúc môn.
  - *Thi trắc nghiệm kết thúc học phần (Rubric A4.1):* **30%**.

2. Tổng quan Đề tài nhóm: OISMS

- **Tên đầy đủ: OISMS** — *Omnichannel Inventory and Sales Management System* (Hệ thống Quản lý Bán hàng và Tồn kho Đa kênh).
- **Bài toán cốt lõi hệ thống giải quyết:**
  1. **Chống bán vượt tồn (Anti-Oversell):** Đồng bộ tồn kho thời gian thực giữa điểm bán trực tiếp (POS tại quầy) và các sàn thương mại điện tử (Shopee, TikTok Shop, Lazada).
  2. **Sổ cái kho bất biến (Append-Only Inventory Ledger):** Mọi biến động kho đều ghi nhận giao dịch thêm mới, ngăn chặn tuyệt đối việc sửa/xóa lịch sử kho bằng trigger CSDL.
  3. **Tự động hóa tính giá vốn bình quân (COGS / WAC):** Chốt giá vốn bình quân gia quyền ngay khi duyệt phiếu nhập kho nguyên tử (Atomic Goods Receipt).
  4. **Kiến trúc SaaS Multi-tenant:** Dùng chung 1 CSDL PostgreSQL 16 nhưng cô lập dữ liệu 100% giữa các cửa hàng bằng EF Core Global Query Filter.

II. CHI TIẾT KHỐI LƯỢNG ĐÓNG GÓP CỦA SINH VIÊN NHỰT PHÙNG MINH

Theo yêu cầu của Giảng viên: *"Tất cả thành viên phải nắm rõ topic và phân định cụ thể phần mình làm là gì"*, toàn bộ sản phẩm của sinh viên **Nhựt Phùng Minh** được liên kết trực tiếp với 3 mã công việc Jira chính thức và kết quả mã nguồn thực tế trong Git repository:

Mã Task Jira	Phân hệ & Tên nhiệm vụ	Chi tiết kỹ thuật & Thuật toán cài đặt	Sản phẩm bàn giao & Bằng chứng
OISMS-16	**[DOC-02] Bối cảnh & Khảo sát Hệ thống** *(Chương I Báo cáo)*	• Phân tích thực trạng bối cảnh bán lẻ đa kênh tại Việt Nam. • Xây dựng **Ma trận phân tích cạnh tranh** chi tiết so sánh OISMS với 4 phần mềm lớn: *KiotViet, Sapo, Haravan, Odoo*. • Xác định khoảng trống thị trường (Gap Analysis): OISMS vượt trội ở cơ chế sổ kho bất biến và chống bán âm phân tán.	• Xuất bản trang Confluence `983053`. • Lưu trữ file: `docs/CONFLUENCE_I_PROJECT_INTRODUCTION.md`. • Trạng thái: **Done**.
OISMS-27	**[BE-02] Xây dựng API Lõi & Logic Nghiệp vụ** *(Backend 2 Lead)*	• Xây dựng **6/10 Controllers** và **28/46 Endpoints API** (chiếm 60.9% toàn bộ API Backend): 1. `AuthController` (5 API): Đăng ký Tenant, Login JWT, Token Rotation, Logout, Me. 2. `CategoriesController` (4 API): Cây danh mục đa tầng. 3. `BrandsController` (4 API): Quản lý thương hiệu. 4. `ProductsController` (10 API): Sản phẩm, SKU, Bảng giá. 5. `SuppliersController` (3 API): Nhà cung cấp. 6. `PosCatalogController` (2 API): Tra cứu/Barcode POS. • Cài đặt **3 Thuật toán nghiệp vụ cốt lõi**: - *Cycle Detection:* Chống vòng lặp cây danh mục. - *EAN-13 Modulo 10:* Tự sinh mã vạch quốc tế. - *JSONB Canonical Signature:* Chống trùng biến thể SKU.	• PR #4 gộp vào `develop`. • Đạt **194/194 bài kiểm thử** tự động (103 Unit + 91 Integration tests) trên PostgreSQL thật. • Code Coverage: **94.83%**. • Trạng thái: **Done**.

Mã Task Jira	Phân hệ & Tên nhiệm vụ	Chi tiết kỹ thuật & Thuật toán cài đặt	Sản phẩm bàn giao & Bằng chứng
OISMS-31	**[DOC-03] Ma trận Phân quyền Màn hình** *(Mục 3.1.3 Báo cáo)*	• Xây dựng **Ma trận phân quyền RBAC cho 39 màn hình** (Owner, Staff, Cashier, Guest). • Đối chiếu và kiểm soát 2 tầng bảo vệ: Route Guard giao diện (chặn đẩy về `/403`) và Backend Controller (`[Authorize(Roles = "...")]`). • Loại bỏ triệt để các màn hình sai lệch (quên mật khẩu, chốt ca độc lập) để khớp 100% với Đề bài và Code thật.	• Xuất bản trang Confluence 3.1.3. • Lưu trữ file: `docs/CONFLUENCE_3_1_3_SCREEN_AUTHORIZATION_MATRIX.md`. • Trạng thái: **Done**.

III. GIẢI MÃ & BÌNH DÂN HÓA 9 KHÁI NIỆM KỸ THUẬT CỐT LÕI BẰNG VÍ DỤ THỰC TẾ

Dưới đây là phần diễn giải tường tận các câu từ súc tích trong tài liệu để giúp bạn hiểu sâu bản chất như việc mua bán ngoài chợ:

1. TỰ ĐỘNG HÓA TÍNH GIÁ VỐN BÌNH QUÂN (COGS / WAC) & PHIẾU NHẬP KHO NGUYÊN TỬ (ATOMIC GOODS RECEIPT)

• **Thuật ngữ chuyên ngành:** `COGS` (Cost of Goods Sold - Giá vốn hàng bán); `WAC` (Weighted Average Cost - Giá vốn bình quân gia quyền); `Atomic` (Nguyên tử - Tất cả hoặc không có gì).

• **Bản chất đời thường:** Khi bạn nhập nhiều đợt hàng với các mức giá nhập khác nhau, thì mỗi món hàng trong kho sẽ được tính giá gốc trung bình là bao nhiêu để sau này bán ra biết mình lời hay lỗ bao nhiêu tiền?

• **Ví dụ minh họa thực tế:**

- *Đợt 1:* Bạn nhập **10 chiếc Áo Polo** giá **100.000 đ/cái** → Tổng vốn đợt 1 = 1.000.000 đ.
- *Đợt 2:* Giá thị trường tăng, bạn nhập thêm **10 chiếc Áo Polo** nữa giá **120.000 đ/cái** → Tổng vốn đợt 2 = 1.200.000 đ.
- Lúc này trong kho có tổng cộng 20 chiếc áo. Giá vốn bình quân WAC được CSDL tự động tính lại ngay lập tức:

```
Giá vốn mới = (10 × 100.000 + 10 × 120.000) / (10 + 10) = 2.200.000 / 20 = 110.000 đ/chiếc
```

• **Chữ "Nguyên tử" (Atomic Goods Receipt) nghĩa là gì?** Quá trình nhập kho gồm 4 bước: (1) Kiểm tra phiếu nháp, (2) Cộng số lượng tồn kho, (3) Tính lại giá vốn WAC, (4) Ghi bút toán sổ cái. Bốn bước này nằm trọn trong 1 Database Transaction: Phải cùng thành công 100%; nếu bước 4 lỡ bị rớt mạng hay cúp điện thì CSDL tự động hủy (Rollback) cả 3 bước trước. Tuyệt đối không bao giờ xảy ra chuyện kho tăng số lượng mà giá vốn chưa kịp tính!

2. CƠ CHẾ SỔ CÁI KHO BẤT BIẾN (APPEND-ONLY INVENTORY LEDGER) & TRIGGER CSDL

• **Thuật ngữ chuyên ngành:** `Append-Only` (Chỉ thêm dòng mới, cấm sửa, cấm xóa).

• **Bản chất đời thường:** Giống hệt cuốn **Sổ sao kê tài khoản ngân hàng**. Tiền của bạn chuyển đi hay nhận về đều được in thành 1 dòng vĩnh viễn trên sao kê, không ai có thể lấy bút xóa đi dòng tiền đó được.

• **Tại sao phải làm vậy?** Nếu phần mềm cho phép bấm nút "Sửa số lượng trong kho từ 10 thành 8", nhân viên kho có thể lén ăn cắp 2 cái áo rồi sửa số tồn mà không ai truy ra được. Hệ thống OISMS cấm tuyệt đối lệnh `UPDATE` và `DELETE` trên bảng sổ cái bằng Trigger PostgreSQL `oisms_deny_mutation`. Muốn giảm 2 cái áo, bắt buộc phải tạo 1 dòng giao dịch xuất kho mới (`-2`) kèm lý do rõ ràng (xuất bán hoặc xuất hủy).

3. CHỐNG BÁN VƯỢT TỒN (ANTI-OVERSELL) & TỒN KHO KHẢ DỤNG (AVAILABLE = ONHAND - RESERVED)

• **Thuật ngữ chuyên ngành:** `OnHand` (Tồn thực tế trên kệ); `Reserved` (Tồn giữ chỗ); `Available` (Tồn khả dụng được bán).

• **Bản chất đời thường:** Tránh tình trạng *"Kho còn đúng 1 cái áo duy nhất, nhưng khách trên Shopee vừa bấm mua, khách tại quầy POS cũng bấm mua cùng lúc"* → Không có hàng giao, bị khách chửi và bị sàn Shopee phạt vi phạm.

• **Ví dụ minh họa thực tế:**

- Trong kho có **OnHand = 5 chiếc áo**.
- Khách trên Shopee vừa đặt mua **3 chiếc**. Hệ thống lập tức khóa giữ chỗ: `Reserved = 3`.
- Tại quầy thu ngân POS, nhân viên tra cứu sản phẩm chỉ thấy: `Available = 5 - 3 = 2 chiếc`. Thu ngân không thể bấm bán chiếc thứ 3 vì nút bấm đã tự động bị khóa (như bạn vừa thấy chiều nay: *Hết tồn kho (0)*).

4. CƠ CHẾ IDEMPOTENCY & POSTGRESQL ADVISORY LOCK

• **Thuật ngữ chuyên ngành:** `Idempotency` (Tính lũy đẳng - bấm 10 lần kết quả vẫn là 1); `Advisory Lock` (Khóa phân tán trong CSDL).

• **Bản chất đời thường:** Hiện tượng mạng lag khi mua hàng online. Khách bấm nút "Thanh toán" nhưng màn hình xoay vòng vòng, khách sốt ruột **bấm liên tiếp 5 lần** vào nút thanh toán.

• **Giải pháp của hệ thống:** Nếu không có Idempotency, tài khoản khách sẽ bị trừ tiền 5 lần và trừ kho 5 lần! Nhóm bạn sử dụng mã `RequestKey` kèm khóa `Advisory Lock`: Khi 5 request cùng gửi lên, CSDL bắt xếp hàng lần lượt. Request đầu tiên trừ kho thành công, 4 request sau nhận diện trùng RequestKey nên trả về ngay kết quả cũ mà không trừ kho thêm lần nào nữa.

5. THUẬT TOÁN CHỐNG VÒNG LẶP CÂY DANH MỤC (CYCLE DETECTION / ANCESTOR TRAVERSAL) [NHỰT PHỤ TRÁCH]

• **Thuật ngữ chuyên ngành:** `Cycle` (Chu trình lặp vô tận); `Ancestor Traversal` (Duyệt ngược dòng tổ tiên).

• **Bản chất đời thường:** Nghịch lý gia phả: *"Một đứa con không thể tự biến thành ông nội hoặc cha của chính bố nó"*.

• **Ví dụ minh họa thực tế:**

- Cây danh mục chuẩn: `Thời trang nam` (Ông nội) → `Áo nam` (Bố) → `Áo Polo` (Con).
- Một nhân viên lỡ tay bấm sửa danh mục cha của `Thời trang nam` thành `Áo Polo`!
- Nếu hệ thống không kiểm tra, vòng lặp luẩn quẩn xuất hiện: Thời trang nam → Áo nam → Áo Polo → Thời trang nam... Khi hiển thị menu trên web, máy chủ sẽ bị treo vĩnh viễn và sập web (Stack Overflow).
- **Thuật toán của bạn (Nhựt):** Dùng vòng lặp `while` lần ngược từ `ParentId` mới lên các đời cha ông. Nếu gặp lại chính danh mục đang sửa → Chặn đứng ngay và báo lỗi: *"Không thể chuyển danh mục thành con của chính con cháu nó"*.

6. CHUẨN HÓA CHỮ KÝ TỔ HỢP (CANONICAL COMBINATION SIGNATURE) TRÊN THUỘC TÍNH ĐỘNG JSONB [NHỰT PHỤ TRÁCH]

• **Thuật ngữ chuyên ngành:** `JSONB` (Lưu trữ JSON nhị phân); `Canonical Signature` (Chữ ký chuẩn hóa duy nhất).

• **Bản chất đời thường:** Chiếc áo Polo có 2 thuộc tính: Màu sắc và Kích cỡ.

- Nhân viên A nhập: `{"Màu": "Đen", "Size": "L"}`.
- Nhân viên B tháng sau nhập: `{"Size": "L", "Màu": "Đen"}`.
- Máy tính thông thường nhìn vào sẽ thấy chuỗi ký tự "Màu trước" và "Size trước" là 2 chuỗi khác nhau, nên cho phép tạo trùng lặp!
- **Thuật toán của bạn (Nhựt):** Tự động sắp xếp chữ cái theo thứ tự bảng chữ cái A-B-C (chữ `C` - Color luôn đứng trước chữ `S` - Size) và đổi thành chữ thường: `o:{color:s:đen,size:s:l}`. Nhờ chữ ký duy nhất này, CSDL nhận diện được ngay là cùng một biến thể và chặn đứng việc tạo trùng.

7. THUẬT TOÁN SINH MÃ VẠCH CHUẨN QUỐC TẾ EAN-13 (MODULO 10) [NHỰT PHỤ TRÁCH]

• **Bản chất đời thường:** Mã vạch 13 chữ số in trên bao bì sản phẩm mà máy quét laser ở siêu thị quét "tít" một cái.

- 12 chữ số đầu: Là mã quốc gia, mã doanh nghiệp và mã sản phẩm.
- **Chữ số thứ 13 (Check Digit - Số kiểm tra):** Tuyệt đối không được gõ bừa, mà phải tính theo công thức Modulo 10 của tổ chức GS1 quốc tế (lấy tổng các vị trí lẻ nhân 1 cộng tổng các vị trí chẵn nhân 3 rồi lấy phần dư chia cho 10).
- **Tác dụng:** Khi máy quét laser đọc mã, máy tự tính lại số thứ 13 xem có khớp không. Nếu khớp máy mới nhận hàng, giúp chống tình trạng máy quét đọc nhầm mã vạch bị mờ.

8. KIẾN TRÚC SAAS MULTI-TENANT & GLOBAL QUERY FILTER

• **Bản chất đời thường:** Giống như **Tòa nhà Chung cư cao cấp**:

- Tất cả các Cửa hàng (Shop A, Shop B, Shop C) đều ở chung một tòa nhà (dùng chung 1 CSDL PostgreSQL).
- Mỗi Shop có một chìa khóa phòng riêng (`TenantId`).
- **Global Query Filter:** Giống như chú bảo vệ đứng ở cửa từng phòng. Bất kỳ câu lệnh truy vấn nào của Shop A cũng tự động bị gắn thêm điều kiện: `WHERE tenant_id = 'Shop_A'` . Chủ Shop A không bao giờ nhìn thấy dữ liệu doanh thu hay kho hàng của Shop B.

9. JWT ACCESS TOKEN & REFRESH TOKEN ROTATION [NHỰT PHỤ TRÁCH]

• **Bản chất đời thường:** Giống như bạn đi chơi ở **Công viên Nước**:

- **Access Token (60 phút):** Giống như chiếc **vòng tay giấy** đeo ở cổ tay. Cứ đến mỗi máng trượt, bạn chìa vòng tay ra là được chơi ngay (rất nhanh, không cần xuất trình căn cước công dân). Nhưng vòng tay chỉ có giá trị 60 phút.
- **Refresh Token (7 ngày):** Giống như **tờ vé gốc có dấu mộc đỏ** cất trong tủ đồ. Khi vòng tay hết hạn, bạn cầm vé gốc ra quầy lễ tân để đổi vòng tay mới.
- **Token Rotation (Xoay tua token):** Mỗi lần đổi vòng tay mới, nhân viên lễ tân lập tức **xé hủy tờ vé gốc cũ và phát tờ vé gốc mới**. Nếu kẻ xấu lượm được vé cũ của bạn đem đi đổi, lễ tân kiểm tra thấy vé đã bị hủy → Lập tức khóa tài khoản lại vì phát hiện có kẻ gian trộm vé (chống tấn công Replay Attack).

IV. BỘ CÂU HỎI VẤN ĐÁP KỸ THUẬT & KỊCH BẢN TRẢ LỜI ĐẠT ĐIỂM A

Dưới đây là **7 câu hỏi trọng tâm** mà giảng viên khó tính hay hỏi xoáy vào phần việc của sinh viên Nhựt Phùng Minh, kèm kịch bản đối đáp lưu loát:

Câu 1: "Em hãy giải thích cơ chế phân quyền RBAC và cách bảo vệ hệ thống của em?"

> **Kịch bản trả lời chuẩn:**
> *"Dạ thưa Thầy, hệ thống OISMS áp dụng mô hình phân quyền RBAC kết hợp kiểm soát 2 tầng bảo vệ:*
> 1. **Lớp 1 - Frontend Route Guard:** Tại component `ProtectedRoute.tsx`, hệ thống kiểm tra JWT Claims. Nếu vai trò `Staff` cố tình truy cập vào `/reports` (Báo cáo tài chính) hoặc `/organization` (Quản trị chi nhánh), Route Guard sẽ chặn lại và đẩy về trang lỗi `/403 Access Denied`. Đối với vai trò `Cashier` (Thu ngân), hệ thống cô lập hoàn toàn chỉ cho phép truy cập Web POS (port 5174), chặn toàn bộ Admin Dashboard.
> 2. **Lớp 2 - Backend API Guard:** Ngay cả khi Client bypass qua Postman/Swagger, toàn bộ 10 Controller đều gắn thuộc tính `[Authorize(Roles = "...")]`. Ví dụ: API xuất báo cáo chỉ cấp quyền cho `Owner`; thu ngân gọi API kho sẽ nhận ngay mã HTTP `403 Forbidden`."

Câu 2: "Tại sao em dùng Refresh Token Rotation mà không dùng JWT đơn thuần?"

> **Kịch bản trả lời chuẩn:**
> *"Dạ thưa Thầy, nếu chỉ dùng JWT đơn thuần với thời hạn dài, khi Token bị lộ qua tấn công XSS hoặc bắt lén gói tin thì kẻ xấu sẽ chiếm toàn quyền cho đến khi token hết hạn. Do đó em thiết kế cặp Token: Access Token 60 phút và Refresh Token 7 ngày áp dụng Token Rotation. Mỗi khi Client gọi API refresh, hệ thống thu hồi token cũ ngay lập tức (`RevokedAt = Now`) và cấp một cặp token mới toanh. Nếu kẻ xấu cố dùng lại token cũ, hệ thống phát hiện hành vi Replay Attack và từ chối ngay với mã `401 Unauthorized`."*

Câu 3: "Thuật toán Chống vòng lặp Cây danh mục (Cycle Detection) của em hoạt động thế nào?"

> **Kịch bản trả lời chuẩn:**
> *"Dạ thưa Thầy, trong hệ thống bán lẻ, danh mục sản phẩm có cấu trúc phân cấp cha - con nhiều tầng. Em giải quyết bài toán chống chu trình lặp vô tận bằng 2 bước:*
> 1. **Chiếm khóa phân tán:** Gọi `SELECT pg_advisory_xact_lock(...)` trên PostgreSQL theo key `tenantId/category_tree` để chặn race condition khi 2 admin cùng sửa cây đồng thời.
> 2. **Duyệt ngược dòng (Ancestor Traversal):** Dùng vòng lặp `while` truy vấn ngược từ `ParentId` mới lên các đời tổ tiên. Nếu bất kỳ tổ tiên nào trùng với `Id` của danh mục đang sửa, hệ thống phát hiện chu trình lặp và lập tức ngắt giao dịch, trả về mã `400 Bad Request` kèm thông báo rõ ràng."*

Câu 4: "Làm thế nào em sinh được mã vạch EAN-13 chuẩn quốc tế hợp lệ?"

> **Kịch bản trả lời chuẩn:**
> *"Dạ thưa Thầy, mã EAN-13 gồm 13 chữ số, trong đó 12 chữ số đầu là dữ liệu mã hóa và chữ số thứ 13 là số kiểm tra (Check Digit) bắt buộc phải tuân theo thuật toán Modulo 10 của tổ chức GS1 quốc tế. Em cài đặt thuật toán tính tổng trọng số chẵn lẻ và tìm phần bù trong `BarcodeHelper.cs`, giúp máy quét laser tại quầy POS quét mã với độ chính xác tuyệt đối, không bao giờ bị lỗi sai lệch mã vạch."*

Câu 5: "Thuộc tính động lưu JSONB, làm sao em chống được việc người dùng tạo trùng biến thể SKU?"

> **Kịch bản trả lời chuẩn:**
> *"Dạ thưa Thầy, một sản phẩm có thể có nhiều thuộc tính tùy biến như Màu sắc, Kích cỡ. Nếu lưu JSON thông thường, chuỗi* `{'size': 'M', 'color': 'Đen'}` *và chuỗi* `{'color': 'Đen', 'size': 'M'}` *sẽ bị coi là khác nhau dù cùng một biến thể. Em đã xây dựng cơ chế* ***Canonical Combination Signature*** *trong* `SkuAttributeHelper.cs` *: Tự động trích xuất các cặp Key-Value, chuẩn hóa chữ thường, sắp xếp Key theo thứ tự Alphabet và gắn tiền tố kiểu dữ liệu (ví dụ:* `o:{color:s:đen,size:s:m}` *). CSDL áp dụng Unique Index trên cặp* `(ProductId, CombinationSignature)` *, chặn đứng 100% việc tạo trùng biến thể SKU."*

Câu 6: "Dữ liệu giữa các Shop (Tenant) có nguy cơ bị đọc nhầm hoặc ghi đè của nhau không?"

> **Kịch bản trả lời chuẩn:**
> *"Dạ hoàn toàn không thưa Thầy. Hệ thống của em áp dụng cơ chế bảo vệ đa người thuê 2 chiều:*
> • **Chiều đọc (Read):** 18 thực thể nghiệp vụ đều được gắn **EF Core Global Query Filter**: Tự động chèn thêm điều kiện `WHERE tenant_id = @currentTenantId` vào tất cả câu lệnh SQL SELECT sinh ra, lập trình viên không thể quên lọc.
> • **Chiều ghi (Write):** Tại `TenantGuard (SaveChangesAsync)` , hệ thống kiểm tra nếu bản ghi bị thay đổi `TenantId` hoặc có khóa ngoại trỏ sang Tenant khác sẽ lập tức ném ngoại lệ `SecurityException` và ngắt kết nối ngay lập tức."*

Câu 7: "Em đã kiểm thử sản phẩm như thế nào để chứng minh hệ thống hoạt động đúng?"

> **Kịch bản trả lời chuẩn:**
> *"Dạ thưa Thầy, nhóm em xây dựng bộ kiểm thử tự động toàn diện gồm* ***194 bài test tự động (Passed 100%)****:*
> • **103 Unit Tests:** Kiểm thử thuật toán băm mật khẩu BCrypt, EAN-13, Canonical Signature, tính giá vốn WAC.
> • **91 Integration Tests:** Chạy trực tiếp trên WebApplicationFactory và container PostgreSQL thật, kiểm chứng triệt để cô lập đa người thuê, chống bán vượt tồn và tính toàn vẹn của sổ cái kho.
> Đồng thời, toàn bộ hệ thống đã được đóng gói Docker Compose gồm đủ 6 container chạy live: PostgreSQL, Redis, PgAdmin, Backend API, Web Admin và Web POS PWA."*

V. QUY TRÌNH THAO TÁC LIVE DEMO TRÊN LỚP (3 PHÚT ĂN ĐIỂM TUYỆT ĐỐI)

Khi Thầy yêu cầu: *"Mở máy lên chạy thử sản phẩm cho Thầy xem"*, sinh viên Nhựt Phùng Minh thực hiện tuần tự 4 bước sau:

1. **Bước 1: Chứng minh môi trường Container chuẩn hóa (30 giây)**
   - Mở Terminal trên máy Mac gõ: `docker ps` .
   - Chỉ cho Thầy thấy đủ 6 dịch vụ đang Up & Healthy: `oism_backend` (port 5000), `oism_admin_dashboard` (port 5173), `oism_pos_app` (port 5174), `oism_postgres` (port 5433), `oism_redis` (port 6379), `oism_pgadmin` (port 5050).
2. **Bước 2: Demo Backend Swagger API do bạn phụ trách (45 giây)**
   - Mở trình duyệt vào link: `http://localhost:5000/swagger` .
   - Chỉ cho Thầy thấy 6 Controller của bạn: `/api/auth` , `/api/categories` , `/api/brands` , `/api/products` , `/api/suppliers` , `/api/pos` .
   - Bấm Execute thử API `POST /api/auth/login` với tài khoản Owner để Thầy thấy Access Token và Refresh Token được sinh ra chuẩn mực.
3. **Bước 3: Demo Phân quyền RBAC trên Admin Dashboard (45 giây)**
   - Vào `http://localhost:5173/login` .

- Bấm nút mẫu đăng nhập bằng **Owner** → Thấy xuất hiện đầy đủ menu Báo cáo và Tổ chức.
- Đăng xuất, bấm nút mẫu đăng nhập bằng **Staff** → Menu Báo cáo lập tức bị ẩn. Gõ trực tiếp link `/reports` → Hệ thống lập tức bật màn hình chặn lỗi `403 Access Denied`.

4. **Bước 4: Demo Quầy bán lẻ Web POS PWA & Cơ chế Chống bán âm (60 giây)**
   - Vào `http://localhost:5174`.
   - Gõ tìm kiếm `Polo` → Cho Thầy thấy sản phẩm hiển thị tồn khả dụng theo đúng chi nhánh.
   - Thêm vào giỏ 10 món Áo Polo (2.990.000 đ), bấm nút **THANH TOÁN (F4)**.
   - Nhập tiền khách đưa 3.000.000 đ → Hệ thống tự động tính tiền thừa 10.000 đ. Bấm xem trước Hóa đơn bán lẻ tiêu chuẩn.

> **KẾT LUẬN & CAM KẾT:**
> *Toàn bộ mã nguồn, tài liệu thiết kế và kết quả kiểm thử trên đây đều được thực hiện thực tế, trung thực và minh bạch 100% trong Git repository của dự án OISMS. Sinh viên Nhựt Phùng Minh hoàn toàn làm chủ kiến thức và sẵn sàng trả lời mọi câu hỏi vấn đáp chuyên sâu của Hội đồng chấm thi.*